

KT 사례기반 AI 교육

- VOC 고객 해지 예측 모델링

1. AIDU 기본 사용법 및 데이터 분석 준비하기(완료)

2. 데이터 탐색하기(완료)

3. 데이터 전처리하기(완료)

4. 머신러닝 모델 구현

5. 딥러닝 심층신경망 모델 구현

```
In [ ]: # 코드실행시 경고 메시지 무시
import warnings
warnings.filterwarnings(action='ignore')
```

```
In [ ]: # 필요 라이브러리 임포트 하기
import numpy as np
import pandas as pd

import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
```

```
In [ ]: #!pip install seaborn
```

Chapter 4. 머신러닝 모델 구현

1. 데이터 가져오기
2. 데이터 전처리 (완료)
3. Train, Test 데이터셋 분할
4. 데이터 정규화 (MinMaxScaling, StandardScaling)
5. 머신러닝 모델 구현
6. 머신러닝 성능 평가

1. 데이터 가져오기

```
In [ ]: # voc_rcp_practice3.csv 데이터 읽어 들이기
df = pd.read_csv("../voc_rcp_practice3.csv")
```

```
In [ ]: df.head()
```

```
In [ ]: # 80컬럼, 20953 라인
df.info()
```

```
In [ ]: df['trm_yn'].value_counts()
```

2. 데이터 전처리하기 - 완료

3. Train, Test 데이터셋 분할

- ① Feature / Target 데이터 분리
- ② Train / Test 데이터 Set 나누기

① Feature / Target 데이터 분리

- Feature는 X, Target은 y로 분리

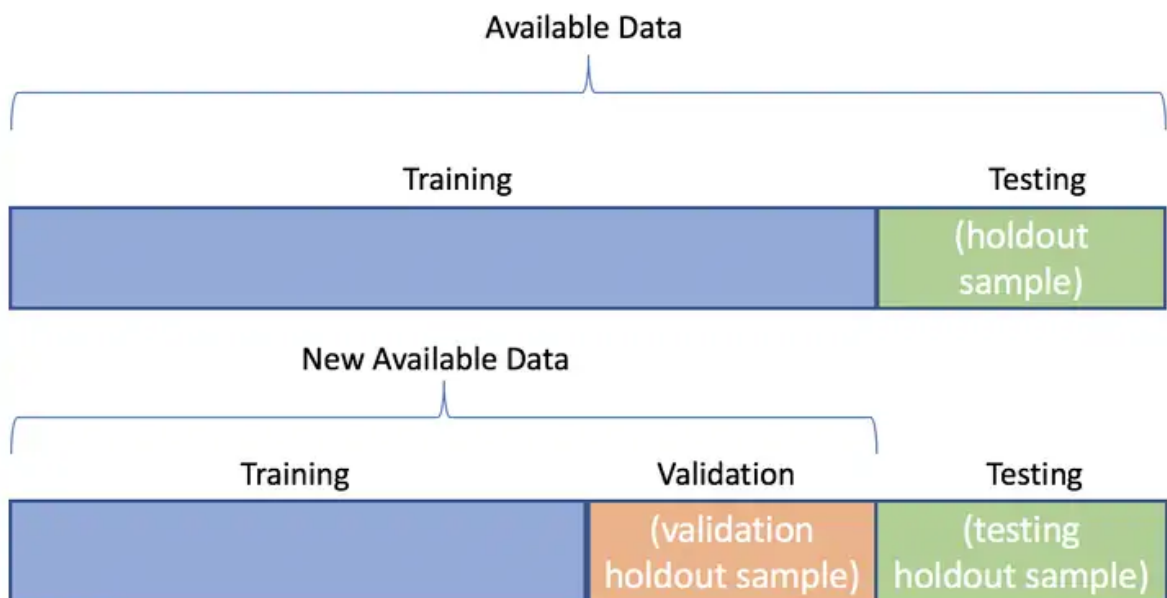
```
In [ ]: # Feature는 Target('trm_yn') 제외한 나머지
X = df.drop(columns=['trm_yn']).values

# Target은 해지여부('trm_yn')
y = df['trm_yn'].values
```

```
In [ ]: print(X.shape, y.shape)
```

② Train / Test 데이터 Set 나누기

- X, y 값을 가지고 7:3 비율로 Train, Test 을 나누세요.
- train_test_split 시 stratify 옵션 사용



[출처] <https://algotrading101.com/learn/train-test-split/>

```
In [ ]: # 라이브러리 임포트
from sklearn.model_selection import train_test_split
```

```
In [ ]: # 데이터 Set 나누기 (Train:Test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, stratify=y,
```

```
In [ ]: print(X_train.shape, y_train.shape)
```

4. 데이터 스케일링(표준화, 정규화)

입력 변수(Feature)들의 값을 일정한 수준으로 맞춰 주는 것은 Feature Scaling 이라 합니다.

데이터 스케일링의 목적은 컬럼 별 차이를 왜곡하지 않고 공통 척도로 변경하기 위함입니다.

가령 아래처럼 나이의 범위는 (0 ~ 100) 이고 , 소득의 범위는 (0 ~ 10,000,000) 이라고 하면 소득은 나이의 약 100,000배이며 범위의 Range도 넓습니다.

이 데이터를 그대로 사용하면 소득을 본질적으로 더 큰 값이기 때문에 나이 피쳐보다 더 큰 영향을 미치게 됩니다.

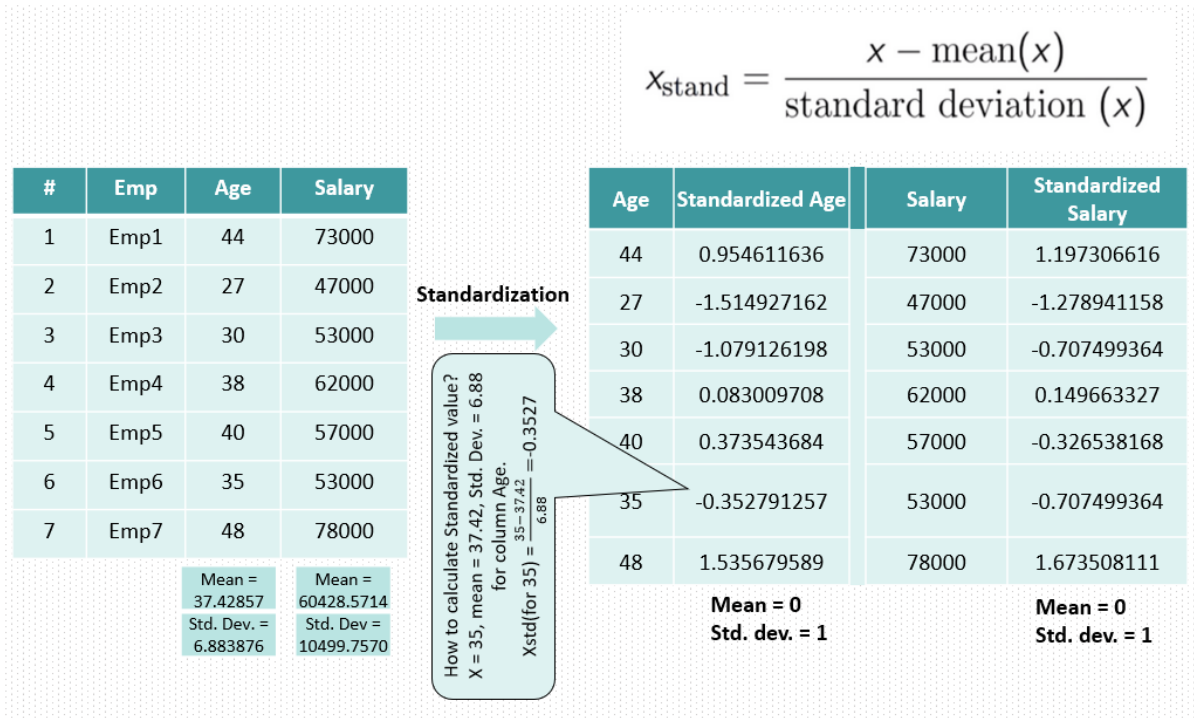
#	Emp	Age	Salary
1	Emp1	44	73000
2	Emp2	27	47000
3	Emp3	30	53000
4	Emp4	38	62000
5	Emp5	40	57000
6	Emp6	35	53000
7	Emp7	48	78000

$$\text{Distance between Emp2 and Emp1} = \sqrt{(27 - 44)^2 + (47000 - 73000)^2} = 31.06$$

$$\text{Distance between Emp2 and Emp3} = \sqrt{(30 - 27)^2 + (53000 - 47000)^2} = 6.70$$

[출처] <https://ashutoshtripathi.com/2021/06/12/what-is-feature-scaling-in-machine-learning-normalization-vs-standardization/>

① Standard Scaler (표준화)



표준정규분포의 속성을 갖도록 피처가 재조정되는 것이다.

0 주위에 표준편차 1의 값으로 배치되도록 피처를 표준화하는 것

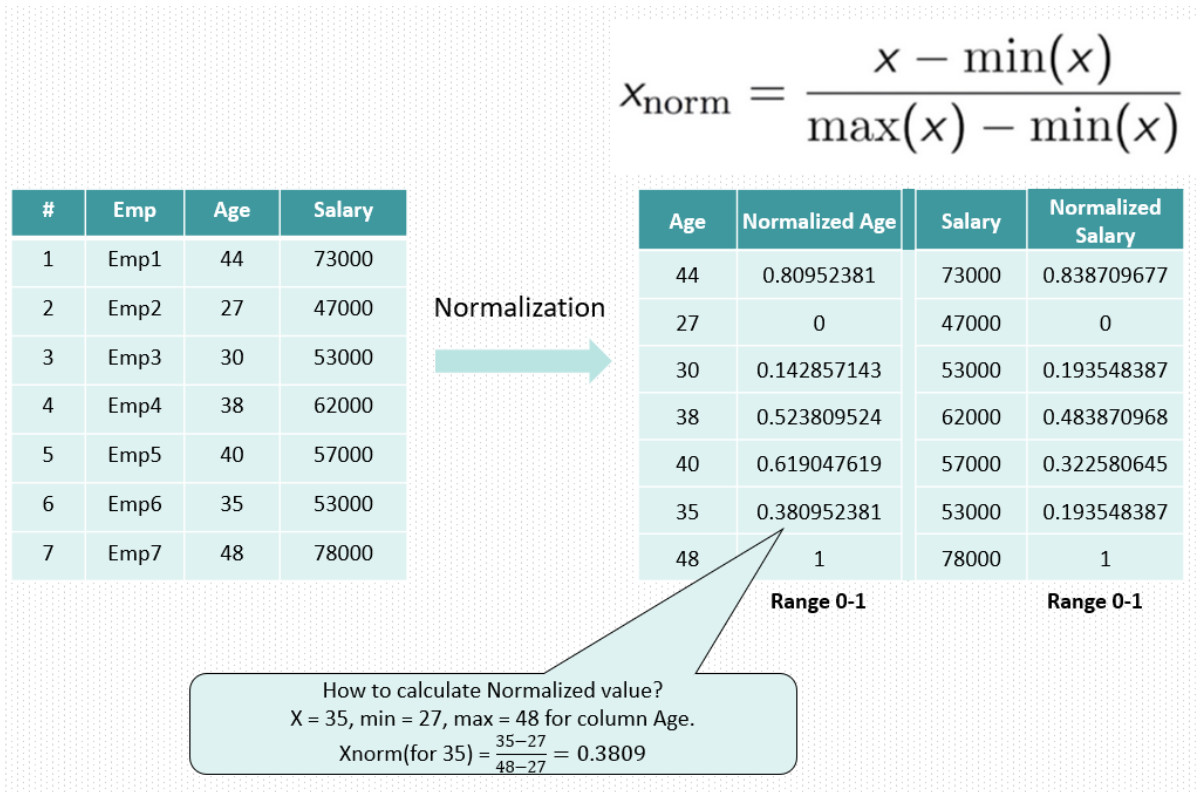
```
In [ ]: # 라이브러리 импорт
from sklearn.preprocessing import StandardScaler
```

```
In [ ]: # 스케일러 호출
scaler = StandardScaler()

# 스케일링 (Train, Test)
X_train_std = scaler.fit_transform(X_train)
x_test_std = scaler.transform(X_test)

# 다른 방법
#scaler.fit(X_train)
#X_train = scaler.transform(X_train)
#x_test = scaler.transform(X_test)
```

② MinMax Scaler (정규화)



Min-Max Scaling은 모든 피처가 정확하게 [0,1] 사이에 위치하도록 데이터를 변경한다. Min-Max Scaling은 모든 피처가 정확하게 [0,1] 사이에 위치하도록 데이터를 변경한다.

```
In [ ]: # 라이브러리 임포트
from sklearn.preprocessing import MinMaxScaler
```

```
In [ ]: # 스케일러 호출
scaler = MinMaxScaler()

# 스케일링 (Train, Test)
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

[실습 1] 머신러닝 준비하기

[Q] 'trm_y'를 y(target)으로 나머지 컬럼을 X(feature)로 만드세요.

target 변수 : y, feature 변수 : X

```
In [ ]: X = df.drop(columns=['trm_yn']).values
y = df['trm_yn'].values
```

[Q] x와 y 값을 가지고 8:2 비율로 Train, Test 데이터 셋을 나누세요.

train 데이터 셋 : X_train, y_train

test 데이터 셋 : X_test, y_test

파라미터 : stratify = y, random_state=42

```
In [ ]: # 라이브러리 임포트
from sklearn.model_selection import train_test_split
```

```
# 데이터 Set 나누기 (Train:Test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, stratify=y,
```

[Q] 사이킷런의 **MinMaxScaler**를 활용하여 **x_train, x_test** 데이터를 정규화 하세요.

```
In [ ]: # 라이브러리 임포트
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

5. 머신러닝 모델 구현

- ① 단일 분류예측 모델 : LogisticRegression, KNN, DecisionTree
- ② 앙상블(Ensemble) 모델 : RandomForest, XGBoost, LGBM, Stacking, Weighted Blending

① 단일 분류예측 모델

- 로지스틱 회귀 (LogisticRegression, 분류)

이진 분류 규칙은 0과 1의 두 클래스를 갖는 것으로, 일반 선형 회귀 모델을 이진분류에 사용할 수 없습니다.

대신 선형 회귀를 시그모이드 함수를 사용하여 로지스틱 회귀 곡선으로 변환할 수 있으며, 로지스틱 회귀 곡선은 0과 1사이에서만 이동할 수 있어 분류에 사용할 수 있습니다.

• 라이브러리 임포트

```
In [ ]: # 라이브러리 임포트
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from sklearn.metrics import classification_report
```

• 모델 학습하기

```
In [ ]: # 모델 호출하기
# 파라미터 (C: 규제강도, max_iter: 계산에 사용할 작업 수)
lr_model = LogisticRegression()

# 모델 학습하기
lr_model.fit(X_train, y_train)
```

• 모델 예측하기

```
In [ ]: lr_pred = lr_model.predict(X_test)
```

- 모델 성능 평가하기

		Predicted Class		
		Positive	Negative	
Actual Class	Positive	True Positive (TP)	False Negative (FN) Type II Error	Sensitivity $\frac{TP}{(TP + FN)}$
	Negative	False Positive (FP) Type I Error	True Negative (TN)	Specificity $\frac{TN}{(TN + FP)}$
		Precision $\frac{TP}{(TP + FP)}$	Negative Predictive Value $\frac{TN}{(TN + FN)}$	Accuracy $\frac{TP + TN}{(TP + TN + FP + FN)}$

- **Precision(정밀도)**

모델이 True라고 분류한 것 중에서 실제 True인 것의 비율
날씨 예측 모델이 맑다고 예측했는데, 실제 날씨가 맑았는지를 나타낸 지표

- **Recall(재현율)**

실제 True인 것 중에서 모델이 True라고 예측한 것의 비율
실제 날씨가 맑은 날 중에서 모델이 맑다고 예측한 비율을 나타낸 지표

- **Accuracy(정확도)**

가장 직관적으로 모델의 성능을 나타낼 수 있는 평가 지표
한달에 맑은 날이 28일이고 비가 오는 날이 2일인 경우,
비가 오는 것을 예측하는 성능을 매우 낮을 수 밖에 없으므로 이를 보완할
지표가 필요합니다.

- **Precision(정밀도)**

정밀도와 재현율의 조화평균입니다.

```
In [ ]: # 오차행렬 (TN FP, FN TP)
print("1.오차 행렬(confusion_matrix)")
print(confusion_matrix(y_test, lr_pred))

# 정확도
print("\n2.정확도(accuracy_score)")
print(accuracy_score(y_test, lr_pred) )

# 정밀도
print("\n3.정밀도(precision_score)")
print(precision_score(y_test, lr_pred) )

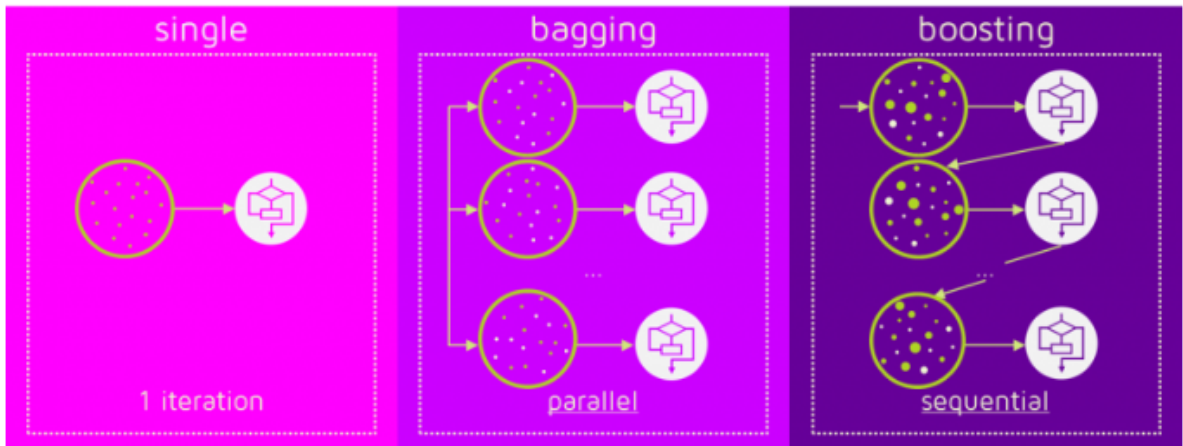
# 재현율
print("\n3.재현율(recall_score)")
print(recall_score(y_test, lr_pred))

# 정밀도 + 재현율
print("\n4.F1 점수")
print(f1_score(y_test, lr_pred) )
```

```
In [ ]: # Classification Report
print(classification_report(y_test, lr_pred))
```

② 앙상블(Essemble) 모델

- 배깅 (Bagging): 여러개의 DecisionTree 활용하고 샘플 중복 생성을 통해 결과 도출.
RandomForest
- 부스팅 (Boosting): 약한 학습기를 순차적으로 학습을 하되, 이전 학습에 대하여 잘못 예측된 데이터에 가중치를 부여해 오차를 보완해 나가는 방식. XGBoost, LGBM



- 랜덤포레스트(RandomForest)

Bagging 대표적인 모델로써, 훈련셋트를 무작위로 각기 다른 서브셋으로 데이터셋을 만들고 여러개의 DecisionTree로 학습하고 다수결로 결정하는 모델

주요 Hyperparameter

- random_state: 랜덤 시드 고정 값. 고정해두고 튜닝할 것!
- n_jobs: CPU 사용 갯수
- max_depth: 깊어질 수 있는 최대 깊이. 과대적합 방지용
- n_estimators: 앙상블하는 트리의 갯수
- max_features: 최대로 사용할 feature의 갯수. 과대적합 방지용
- min_samples_split: 트리가 분할할 때 최소 샘플의 갯수. default=2. 과대적합 방지용

```
In [ ]: # 라이브러리 임포트
from sklearn.ensemble import RandomForestClassifier

# 모델 불러오기
rfc_model = RandomForestClassifier(n_estimators=100)

# 모델 학습하기
rfc_model.fit(X_train, y_train)

# 예측하기
rfc_pred = rfc_model.predict(X_test)

# 성능평가하기
print(classification_report(y_test, rfc_pred))
```

- XGBoost

여러개의 DecisionTree를 결합하여 Strong Learner 만드는 Boosting 앙상블 기법
Kaggle 대회에서 자주 사용하는 모델이다.

주요 특징

- scikit-learn 패키지가 아닙니다.
- 성능이 우수함
- 학습시간이 매우 느리다

주요 Hyperparameter

- random_state: 랜덤 시드 고정 값. 고정해두고 튜닝할 것!
- n_jobs: CPU 사용 갯수
- learning_rate: 학습율. 너무 큰 학습율은 성능을 떨어뜨리고, 너무 작은 학습율은 학습이 느리다. 적절한 값을 찾아야함. n_estimators와 같이 튜닝. default=0.1
- n_estimators: 부스팅 스테이지 수. (랜덤포레스트 트리의 갯수 설정과 비슷한 개념). default=100
- max_depth: 트리의 깊이. 과대적합 방지용. default=3.
- subsample: 샘플 사용 비율. 과대적합 방지용. default=1.0
- max_features: 최대 사용할 feature의 비율. 과대적합 방지용. default=1.0

```
In [ ]: # 패키지 설치
!pip install xgboost
```

```
In [ ]: # 라이브러리 임포트
from xgboost import XGBClassifier

# 모델 불러오기
xgb_model = XGBClassifier(n_estimators=3, random_state=42)

# 모델 학습하기
xgb_model.fit(X_train, y_train)

# 예측하기
xgb_pred = xgb_model.predict(X_test)

# 성능평가하기
print(classification_report(y_test, xgb_pred))
```

[실습 2] 머신러닝 모델 구현하기

[Q] 아래의 조건으로 KNN(K-Nearest Neighbor) 모델을 구현해 보세요.

- 모델은 변수명 knn_model 로 저장하세요.
- 파라미터 n-neighbors는 5로 설정하세요.
- 예측 값은 변수명 knn_pred로 저장하세요.
- 성능 평가는 F1-Score로 하세요.

```
In [ ]: # 라이브러리 임포트
from sklearn.neighbors import KNeighborsClassifier

# 모델 불러오기
knn_model = KNeighborsClassifier(n_neighbors=5)
```

```
# 모델 학습하기
knn_model.fit(X_train, y_train)

# 예측하기
knn_pred = knn_model.predict(X_test)

# 성능평가하기
print(f1_score(y_test, knn_pred))
```

[Q] 아래의 조건으로 결정트리(DecisionTree) 모델을 구현해 보세요.

- 모델은 변수명 dt_model 로 저장하세요.
- 파라미터 max_depth는 10로 설정하세요.
- 예측 값은 변수명 dt_pred로 저장하세요.
- 성능 평가는 정확도로 하세요.

```
In [ ]: # 라이브러리 임포트
from sklearn.tree import DecisionTreeClassifier

# 모델 불러오기
dt_model = DecisionTreeClassifier(max_depth=10)

# 모델 학습하기
dt_model.fit(X_train, y_train)

# 예측하기
dt_pred = dt_model.predict(X_test)

# 성능평가하기
print(accuracy_score(y_test, dt_pred))
```

[Q] 아래의 조건으로 랜덤포레스트(RandomForest) 모델을 구현해 보세요.

- 모델은 변수명 rfc_model 로 저장하세요.
- 파라미터 max_depth는 30, n_estimators는 5로 설정하세요.
- 예측 값은 변수명 rfc_pred로 저장하세요.
- 성능 평가는 classification report 하세요.

```
In [ ]: # 라이브러리 임포트
from sklearn.ensemble import RandomForestClassifier

# 모델 불러오기
rfc_model = RandomForestClassifier(max_depth=30, n_estimators=5, random_state=42)

# 모델 학습하기
rfc_model.fit(X_train, y_train)

# 예측하기
rfc_pred = rfc_model.predict(X_test)

# 성능평가하기
print(classification_report(y_test, rfc_pred))
```

6. 머신러닝 모델 성능 비교하기

① 모델 성능 비교하기

```
In [ ]: performance_acc = [accuracy_score(y_test, lr_pred), accuracy_score(y_test, knn_pred), accuracy_score(y_test, rf_pred)]
performance_precision = [precision_score(y_test, lr_pred), precision_score(y_test, knn_pred), precision_score(y_test, rf_pred)]
performance_recall = [recall_score(y_test, lr_pred), recall_score(y_test, knn_pred), recall_score(y_test, rf_pred)]
performance_f1_score = [f1_score(y_test, lr_pred), f1_score(y_test, knn_pred), f1_score(y_test, rf_pred)]
```

```
In [ ]: performance_index = ['accuracy', 'precision', 'recall', 'f1_score']
performance_columns = ['Logistic', 'KNN', 'Decision', 'RandomForest']

performance_total = pd.DataFrame(data=[performance_acc, performance_precision, performance_recall, performance_f1_score],
                                index=performance_index, columns=performance_columns)
```

```
In [ ]: performance_total
```

```
In [ ]: plt.figure(figsize=(8,4))
plt.title('Accuracy')
sns.barplot(x=performance_columns, y=performance_acc)
plt.show()
```

② 모델 Feature 중요도 비교하기

```
In [ ]: feature = df.drop(columns=['trm_yn'])

rfc_importances_values = rfc_model.feature_importances_
rfc_importances = pd.Series(rfc_importances_values, index=feature.columns)
rfc_top10 = rfc_importances.sort_values(ascending=False)[:10]

plt.figure(figsize=(8,6))
plt.title('Top 10 Feature Importances')
sns.barplot(x=rfc_top10, y=rfc_top10.index)
plt.show()
```

```
In [ ]:
```